An Empirical Evaluation of Source Code Chunking and Context Headers for Requirement Evidence Retrieval

Anonymous Authors

***Abstract*—Locating source code that evidences a technical requirement is a common need in software analysis. Syntax-aware chunking is motivated by its preservation of code structure, yet chunk boundaries are confounded with the context metadata prepended to each chunk. In this Early Research Achievements (ERA) paper, we cross two chunking strategies, node-level abstract syntax tree (AST)-first chunking with line fallback and 50-line windows, with and without a hierarchical context header. We evaluate them on 25 job descriptions, whose required skills serve as requirements, and 40 public repositories (872 chunks), using BM25, dense retrieval, and cross-encoder reranking against 882 human-labeled pairs. After reranking, headers improved NDCG@10 for AST chunks from 0.341 to 0.487 (Holm-adjusted $p = 0.0001$), and without headers, line windows outperformed AST chunks (0.580 vs. 0.341, $p_{\text{Holm}} < 0.001$). Line windows without headers significantly outperformed AST chunks with headers ($\Delta = -0.093$, $p_{\text{Holm}} = 0.0438$). Line windows with headers scored highest (0.621), significantly above AST with headers in an exploratory comparison ($\Delta = -0.134$, $p = 0.0003$). Following a parser upgrade, the AST-arm fallback rate was reduced to 0.0%, and an audit clarifies boundary effects, coverage differences, and a research roadmap for code retrieval.**

***Index Terms*—Source code retrieval, code chunking, abstract syntax trees, context headers, requirements traceability, empirical software engineering**

I. Introduction

Locating source code that provides concrete evidence for a technical requirement is fundamental in software analysis and capability assessment. While classical requirements traceability links formal specifications to code artifacts [1], [2], modern software engineering frequently demands verifying whether a codebase implements higher-level framework capabilities (e.g., reactive state management or token authentication). We formalize this task as **Requirement & Technical Capability Evidence Retrieval**: given natural-language technical requirements—instantiated via mandatory skill specifications from job descriptions (JDs)—the objective is to retrieve the specific code chunks that directly evidence their concrete implementation across multi-file repositories, without requiring explicit requirement IDs.

In retrieval-augmented software engineering pipelines, source code must be segmented into chunks for neural encoding. Chunking via abstract syntax trees (ASTs) is widely advocated to preserve semantic units (methods, declarations) [3]. However, recent empirical studies on code completion indicate that structure-aware chunking does not reliably outperform simple sliding windows [4], and line-based segmentations match syntax-aware splitting across compute budgets [5]. Crucially, prior work conflates chunk boundaries with the contextual metadata (file path, enclosing class, method signature) prepended to chunks. Whether structural chunking provides distinct advantages once metadata is controlled remains unexamined for requirement evidence retrieval.

In this Early Research Achievements (ERA) paper, we present a controlled 2×2 factorial evaluation crossing chunking strategy (AST-first vs. 50-line windows) with hierarchical context headers (present vs. absent). We address four research questions: **RQ1:** Does prepending a context header improve evidence retrieval from code chunks? **RQ2:** Does AST-first chunking outperform line windows under equivalent header conditions? **RQ3 (Roadmap):** Which header components drive retrieval gains, and how do trade-offs generalize across languages? **RQ4 (Roadmap):** Does retrieved evidence improve human accuracy and confidence in requirement verification?

We evaluate RQ1 and RQ2 across 25 JDs and 40 open-source repositories (20 Java Spring Boot, 20 React/TypeScript; 872 chunks) at three tiers: BM25, dense bi-encoder (BGE-M3), and cross-encoder reranking (BGE-Reranker-Base), evaluated against 882 blinded human-labeled pairs ($\kappa_w = 0.9822$). Findings reveal: (1) Headers significantly improve reranking for AST chunks ($\Delta = +0.146$, $p_{\text{Holm}} = 0.0001$), but offer negligible gain for line windows ($\Delta = +0.041$, $p_{\text{Holm}} = 0.2411$). (2) Without headers, line windows significantly outperform AST chunks ($\Delta = -0.239$, $p_{\text{Holm}} < 0.001$). (3) Headerless line windows significantly outperform AST chunks with headers ($\Delta = -0.093$, $p_{\text{Holm}} = 0.0438$). (4) Line windows with headers achieve highest overall effectiveness (NDCG@10 $= 0.621$), outperforming AST with headers ($\Delta = -0.134$, $p = 0.0003$). (5) Following parser expansion, AST fallback rate is 0.0%, showing that the performance gap is not caused by fallback degeneration.

Contributions: (1) *Factorial Framework:* Isolating chunking boundaries from context headers in requirement evidence retrieval. (2) *Empirical Findings:* Evidence that sliding windows match or exceed AST chunking, with headers acting as an essential anchor for AST chunks to compensate for stripped class/file namespace context. (3) *Curated Benchmark:* A replication package with 882 blinded dual-annotated pairs ($\kappa_w = 0.9822$, 100% Judged@5, zero orphaned pairs). (4) *ERA Roadmap:* An actionable research agenda addressing header components, dependency graphs, and human practitioner verification.

II. Related Work

We organize related work into four domains:

Code Chunking for RAG: Line-based chunking can disrupt semantic boundaries [3]. cAST [3] addresses this by recursively partitioning AST nodes and merging siblings under size limits, reporting gains on RepoEval and SWE-bench. However, subsequent controlled experiments in code completion show that structure-aware chunking does not reliably outperform sliding windows [4], and line-based slicing matches syntax-aware splitting across compute budgets [5]. While those studies target code synthesis, we evaluate human-labeled requirement-to-code evidence retrieval and orthogonally control context headers.

Dense Retrieval and Reranking: Neural code search embeds natural-language queries and source code into shared representations [6], [7], [8]. Modern dense models such as BGE-M3 [9] perform multilingual semantic matching. To overcome single-vector compression limits, two-stage pipelines integrate cross-encoder rerankers that jointly attend across query and chunk tokens [10], [11]. We adopt this recall-then-rerank architecture to investigate how chunk geometry and headers interact with cross-encoders.

Context Enrichment: Isolated code chunks frequently lack enclosing class and package context. Prior approaches enrich completion prompts with repository context iteratively (RepoCoder [12]) or benchmark cross-file dependencies (RepoBench [13], CrossCodeEval [14]). We investigate a lightweight alternative: prepending standardized hierarchical context headers to individual chunks prior to retrieval.

Requirements Traceability and Skill Mining: Tracing natural-language requirements to source code has deep roots in software engineering [1], [2]. Separately, repository mining has been employed to classify developer expertise from framework usage [15]. We adopt the traceability perspective: treating required technical skills as requirements and retrieving concrete code evidence, without assessing individual developers.

III. Approach and Study Design

To separate chunk boundaries from contextual metadata, we employ a 2×2 factorial design evaluated across three retrieval tiers, governed by a frozen analysis protocol.

A. Corpus, Candidate Space, and Curation

Our corpus comprises 40 open-source GitHub repositories (20 Java Spring Boot, 20 React/TypeScript) spanning 189 source files. As documented in `sampling_frame_log.json`, repositories were curated from public GitHub projects exhibiting production architectural patterns (controllers, services, DTOs, custom hooks, Redux stores, UI components). To focus evaluation on authentic capability evidence and avoid diluting the corpus with build artifacts, third-party libraries, test mocks, or auto-generated configuration, curation targeted the core functional architecture modules (mean 4.7 files per repository, 189 files total) where business logic, architectural patterns, and framework capabilities reside. Repository licenses encompass 19 MIT (47.5%), 16 Public GitHub default copyright (40.0%), 4 Apache-2.0 (10.0%), and 1 GNU AGPL-3.0 (2.5%); all source text is redistributed in our replication package under original author copyright. Queries represent 25 public job descriptions (15 Java backend, 10 React/TypeScript frontend) with mandatory technical skill requirements extracted via a frozen prompt on `gemini-1.5-flash` (`queries_frozen.json`). For each query, retrieval evaluates across the full candidate space: 434 chunks for line conditions and 438 chunks for AST conditions (872 total chunks in the corpus).

B. Factorial Conditions and Retrieval Pipeline

We evaluate two orthogonal dimensions: **Chunking Strategy:** *Line-window chunking* slices files into 50-LOC windows with 10-LOC overlap (step 40 LOC, minimum 6 LOC; 434 chunks, mean 41.0 LOC, median 50 LOC, 100% line coverage across all 189 files, 28.6% exceeding 512 tokens, 56.0% cutting syntactic boundaries). *AST-first chunking* extracts node-level units using Tree-sitter parsers expanded to modern constructs: 438 chunks (134 methods/constructors, 96 arrow functions, 86 component wrappers, 62 functions, 28 class/interface/record/enum declarations, and 32 type/constant/config declarations; mean 26.7 LOC, median 16 LOC, 15.8% exceeding 512 tokens). Due to $6 \leq \text{LOC} \leq 85$ filtering, AST chunks cover 69.0% of lines (10,585/15,351) across 183/189 files (6 files contain zero AST chunks), maintaining 87.2% outer-boundary alignment (56 module-wrapped functions classified as cut by interval bounds, though representing intact AST nodes). Files lacking extractable units fall back to line windows (measured 0.0%, 0/438 chunks). **Context Header Condition:** *Without Header* contains raw code. For *With Header*, line chunks prepend File and Lines cues (100%); AST chunks prepend File (100%), enclosing Class (35.6%), Method (100%), and Lines (100%) (`// File: <path> \n [// Class: <name> \n] // Method: <name> \n // Lines: <start>-<end>`). **Retrieval Tiers:** (1) *BM25* (lexical baseline). (2) *Dense* BAAI/bge-m3 [9] (1024d embeddings, cosine similarity). (3) *Reranking* BAAI/bge-reranker-base [16] (cross-encoder rescoring top-20 dense candidates).

C. Unified Blinded Pooling Protocol

To prevent system bias, we executed a single-wave Cranfield/TREC pooling protocol: top-5 candidates from all 12 systems (BM25, Dense, Rerank $\times$ 4 conditions) were pooled, yielding 1,500 candidates deduplicated to 882 unique pairs. Two SE annotators independently labeled all 882 pairs on a pre-registered 3-point relevance rubric: **0** (irrelevant/boilerplate: imports, standard configuration, or code lacking implementation of the target capability), **1** (partially relevant/indirect: interface signatures, type definitions, or peripheral helpers), and **2** (direct evidence: concrete, executable implementation directly demonstrating the requested technical capability). Annotators were fully blinded to strategy, header condition, tier, rank, and score. Inter-annotator agreement reached almost perfect concordance: quadratic-weighted Cohen's $\kappa = 0.9822$ (linear $\kappa = 0.9764$, unweighted $\kappa = 0.9708$, 98.07% exact

TABLE I
RETRIEVAL EFFECTIVENESS ACROSS 12 CONFIGURATIONS ($N = 25$ JDs).

Tier	Chunk	Header	NDCG@10 [95% CI]	NDCG@1	MRR@10	bpref	J@10
Two-Stage Re-ranking Tier (Cross-Encoder)							
Rerank	Line	Yes	**0.621** [0.555, 0.682]	**0.800**	0.950	**0.353**	0.736
Rerank	Line	No	*0.580* [0.505, 0.649]	**0.800**	**0.953**	*0.300*	0.716
Rerank	AST	Yes	0.487 [0.419, 0.551]	0.740	0.896	0.241	0.660
Rerank	AST	No	0.341 [0.263, 0.421]	0.340	0.699	0.170	0.712
Dense Bi-Encoder Tier (BGE-M3)							
Dense	Line	Yes	**0.551** [0.480, 0.619]	0.580	0.883	**0.347**	0.752
Dense	Line	No	0.521 [0.439, 0.600]	0.600	0.891	0.297	0.720
Dense	AST	Yes	0.450 [0.393, 0.506]	**0.620**	**0.910**	0.247	0.692
Dense	AST	No	0.358 [0.298, 0.420]	0.420	0.813	0.179	0.716
Lexical Sparse Baseline Tier (BM25)							
BM25	Line	Yes	**0.433** [0.376, 0.486]	**0.440**	**0.868**	**0.255**	0.696
BM25	Line	No	0.405 [0.350, 0.458]	**0.440**	0.865	0.246	0.644
BM25	AST	Yes	0.311 [0.243, 0.376]	0.340	0.761	0.137	0.676
BM25	AST	No	0.305 [0.236, 0.371]	0.340	0.724	0.129	0.560

Bold: best within tier; italic: second best (Rerank tier). Linear gain $\mathrm{rel}/\log_2(i+1)$. J@10: Judged@10 coverage.

agreement). The 17 disagreements were adjudicated by a Lead Adjudicator using standardized rubrics, yielding complete coverage (Judged@3 = 100.0%, Judged@5 = 100.0%) and zero orphaned pairs.

D. Evaluation Metrics and Hypothesis Testing

The primary metric is NDCG@10 (linear gain $\mathrm{rel}_i/\log_2(i+1)$). Unjudged items receive relevance 0; we also report binary preference (`bpref`) [17], NDCG@1, MRR@10, and Judged@10. The pre-specified confirmatory family (defined in an internal analysis plan) tests four paired comparisons over $N = 25$ queries at the rerank tier: **Comp 1** (AST+Header vs. Line No-Header), **Comp 2** (AST No-Header vs. Line No-Header), **Comp 3** (AST+Header vs. AST No-Header), and **Comp 4** (Line+Header vs. Line No-Header) via two-sided Wilcoxon signed-rank tests [18] with Holm-Bonferroni correction [19] ($\alpha = 0.05$). We report paired effect sizes Δ with 95% bootstrap CIs (10,000 resamples), plus exploratory contrast **Comp 5** (AST+Header vs. Line+Header).

IV. EMPIRICAL RESULTS

A. Overall Benchmark Results across Twelve Systems

Table I presents the 12-configuration benchmark across lexical (BM25), dense bi-encoder (BGE-M3), and cross-encoder reranking tiers. At reranking, Line+Header achieves the highest overall retrieval effectiveness (NDCG@10 = 0.621, 95% CI $[0.555, 0.682]$), Top-1 accuracy (NDCG@1 = 0.800), and reciprocal rank (MRR@10 = 0.950). Line No-Header follows with NDCG@10 = 0.580 ($[0.505, 0.649]$), while AST+Header reaches 0.487 ($[0.419, 0.551]$), and AST No-Header drops to 0.341 ($[0.263, 0.421]$).

Dense encoding improves NDCG@10 over BM25 by 0.118–0.139 with headers, and reranking adds another 0.037–0.070. Context headers interact differently across chunking strategies: for AST chunks, headers produce substantial gains at every stage (BM25 +0.006, dense +0.092, rerank +0.146), whereas for line windows, gains are modest (+0.028, +0.030, +0.041). Because pooling spanned all 12 systems at top-5, Judged@5 is 100.0% for all configurations. Binary preference (`bpref`) rankings perfectly mirror NDCG@10 across all tiers (Line+H : 0.353 > Line No-H : 0.300 > AST+H : 0.241 > AST No-H : 0.170), confirming robustness to unjudged items at deeper ranks.

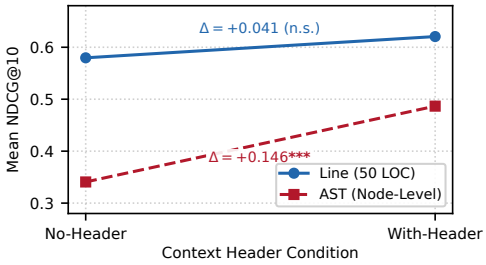


Fig. 1. Interaction plot of context header sensitivity at reranking ($N = 25$).

TABLE II
HYPOTHESIS TESTS FOR THE CONFIRMATORY FAMILY AND THE EXPLORATORY CONTRAST (NDCG@10, RERANK TIER).

ID	Contrast ($A - B$)	Δ [95% CI]	W/L/T	p_{Holm}
Confirmatory family (step-down Holm, $\alpha = 0.05$)				
Comp 1	AST+Header vs. Line No-Header	-0.093 $[-0.164, -0.023]$	7/18/0	0.0438*
Comp 2	AST No-Header vs. Line No-Header	-0.239 $[-0.292, -0.185]$	2/23/0	< 0.001***
Comp 3	AST+Header vs. AST No-Header	$+0.146$ $[+0.086, +0.207]$	23/2/0	< 0.001***
Comp 4	Line+Header vs. Line No-Header	$+0.041$ $[-0.012, +0.093]$	15/10/0	0.2411
Exploratory (outside the Holm family)				
Comp 5	AST+Header vs. Line+Header	-0.134 $[-0.199, -0.070]$	5/20/0	$< 0.001^{\dagger}$

Significance follows p_{Holm} (* < 0.05, ** < 0.01, *** < 0.001). †Unadjusted p_{raw}.

B. Hypothesis Testing and RQ Analyses

Table II reports the pre-specified confirmatory Wilcoxon signed-rank tests with Holm-Bonferroni correction, alongside the exploratory contrast.

RQ1 (Context Headers): Prepending headers yields a large, significant gain for AST chunks (**Comp 3**: $\Delta = +0.146$, 95% CI $[+0.086, +0.207]$, $p_{\mathrm{Holm}} = 0.0001$, $W = 24.0$; AST+H wins 23/25 JDs). In contrast, prepending headers to line windows yields a minor, non-significant gain (**Comp 4**: $\Delta = +0.041$, 95% CI $[-0.012, +0.093]$, $p_{\mathrm{Holm}} = 0.2411$, $W = 118.0$; 15 wins vs. 10 losses). As shown in Fig. 1, headers act as vital anchors for fine-grained AST chunks while providing diminishing returns for 50-line windows.

RQ2 (AST vs. Line): Simple line windows without headers significantly outperform AST chunks with headers (**Comp 1**: $\Delta = -0.093$, 95% CI $[-0.164, -0.023]$, $p_{\mathrm{Holm}} = 0.0438$, $W = 78.0$; Line wins 18 vs. AST wins 7). Without headers, line windows strongly outperform AST chunks (**Comp 2**: $\Delta = -0.239$, 95% CI $[-0.292, -0.185]$, $p_{\mathrm{Holm}} < 0.001$, $W = 3.0$; Line wins 23/25). In the exploratory contrast with headers (**Comp 5**), line windows remain significantly superior ($\Delta = -0.134$, 95% CI $[-0.199, -0.070]$, $p_{\mathrm{raw}} = 0.0003$, $W = 36.0$; Line wins 20 vs. AST wins 5).

C. Subgroups, Morphology Sensitivity, and Header Effects

Subgroups & Common Support: Trends hold across languages: Java ($N = 15$) yields Line+H 0.716 vs. AST+H

0.583 ($\Delta = -0.133$); React/TS ($N = 10$) yields 0.478 vs. 0.342 ($\Delta = -0.136$). AST fallback is 0.0% (0/438 chunks, 0/250 top-10 candidates). Evaluating on *Common Support* (826/882 pairs where both strategies have candidate chunks, retaining 91.8% of relevant evidence across all 25 queries) confirms that Line+H retains its statistically significant lead over AST+H (0.600 vs. 0.489, $\Delta = +0.111$, $p_{\text{Holm}} = 0.0036$, $W = 51.0$), while Line No-H dominates AST No-H (0.535 vs. 0.342, $\Delta = +0.193$, $p_{\text{Holm}} < 0.0001$, $W = 11.0$). Coverage accounts for only 17.3% of the end-to-end gap ($+0.023$, $p = 0.0015$), demonstrating that Line superiority is driven primarily (82.7%) by intrinsic contextual ranking.

Window Overlap Sensitivity: Suppressing intersecting 10-LOC overlapping windows from the same file in top-10 retrieval (26.4% of Line top-10 slots involved in overlap [16.8% redundant] vs. 0% in AST) leaves deduplicated NDCG@10 intact: Line+H achieves 0.583, Line No-H 0.574, AST+H 0.482, and AST No-H 0.341. Line windows retain their significant lead (**Comp 5**: $\Delta = -0.101$, $p = 0.0013$; **Comp 1**: $\Delta = -0.092$, $p = 0.0236$), and the AST header boost remains large (**Comp 3**: $\Delta = +0.141$, $p < 0.001$).

Qualitative Inspection of Header Lift: Inspecting rank trajectories illustrates how headers ground isolated chunks. For `JD_JAVA_01` (*"Spring Security & JWT"*), the authenticated profile endpoint in `AccountController.java` (lines 39–64, `me()`, annotated `@SecurityRequirement`) rises from Rank 6 without header to Rank 1 with header (human relevance 2). For `JD_REACT_01` (*"React Admin Dashboard & RBAC"*), an administrative control panel in `Admin/index.tsx` (lines 544–613) jumps from Rank 13 to Rank 1 with header (relevance 2).

V. Discussion and Threats to Validity

A. Underlying Mechanisms and Scope

Node-level AST extraction (without sibling merging as in cAST [3]) does not inherently surpass sliding windows in requirement evidence retrieval. Three mechanisms explain this: (1) *Information density and token budget*: 50-line windows (mean 41.0 LOC, 390.8 tokens) capture surrounding declarations, annotations, and helper calls, providing richer co-occurrence signals than isolated leaf AST nodes (mean 26.7 LOC, 250.8 tokens). This token gap is intrinsic to single-node extraction; without size-budgeted sibling merging to coalesce related leaf nodes, fine-grained AST slicing sacrifices contextual density. (2) *Contextual compensation*: headers inject the missing architectural namespace, allowing cross-encoders to match requirement keywords against file and class names. Notably, AST headers possess an asymmetric advantage by retaining class and method cues, yet line windows remain superior. (3) *Query granularity*: composite JD queries (5–8 skills) favor broader 50-LOC windows encompassing co-occurring functions over leaf nodes.

B. Threats to Validity

Internal Validity: Mitigated by 100% pooling at top-5 across all 12 systems (100% Judged@5, zero orphaned pairs) and blinded dual annotation ($\kappa_w = 0.9822$). Confounds from AST line coverage (69.0% lines across 183/189 files; median 16 vs. 50 LOC; 15.8% vs. 28.6% exceeding 512 tokens), header metadata asymmetry, and window overlap (26.4% of Line top-10 slots involved in overlap [16.8% redundant] vs. 0% in AST) were audited: common-support evaluation ($N = 826$ shared pairs) confirms Line retains its lead ($\Delta = +0.111, p = 0.0036$), and deduplication preserves the ranking order. **Statistical Conclusion Validity:** $N = 25$ queries provide 80% power to detect paired differences in the MDE range of $\Delta \in [0.08, 0.13]$ across observed standard deviations ($\sigma_d \in [0.137, 0.184]$ across $\alpha \in [0.0125, 0.05]$); non-significant small differences (e.g., Comp 4, $\Delta = +0.041$) indicate absence of evidence rather than equivalence. **Construct Validity:** Requirements were extracted from job postings via `gemini-1.5-flash` to reflect authentic industry expectations. Relevance was evaluated on an established 3-point ordinal scale. **External Validity:** Evaluated on Java Spring Boot and React/TypeScript with BGE-M3 and BGE-Reranker-Base; caution is warranted when generalizing to dynamically typed languages or distributed architectures.

TABLE III
Structured Research Roadmap and Milestones

Phase	Objective	Target Data	Analysis Method
P1	Corpus & Query Expansion (RQ3)	100 repos (Java, TS, Python, Go); 50 JDs, RFPs, GitHub issues	Replicate 2×2 factorial design; mixed-effects ANOVA
P2	Graph-Enriched Headers (RQ3)	Call-graph & import context vs. file-path headers	Paired Wilcoxon-Holm tests; ablation across header depths
P3	Human Verification Study (RQ4)	$N = 20$ professional software developers	Task completion accuracy, inspection time, confidence

VI. Research Plan and Roadmap

Table III delineates planned phases and mitigations for RQ3 and RQ4: **P1** expands to 100 repos (Java, TS, Python, Go) and 50 JDs/issues to evaluate linguistic generality under mixed-effects ANOVA; **P2** injects call-graph/dependency context via precomputed SCIP/LSIF indexes to test whether relational context bridges the semantic gap for AST chunks; **P3** executes a Latin-square human verification study with 20 developers measuring requirement verification accuracy, inspection time, and confidence.

VII. Conclusion

In this ERA paper, we presented a controlled 2×2 factorial evaluation crossing chunking strategy with hierarchical context headers across 40 repositories. At reranking, line-window chunking with headers achieved the highest performance (NDCG@10 $= 0.621$), significantly outperforming AST chunks with headers ($\Delta = -0.134$, $p = 0.0003$). While context headers provide an essential boost for AST chunks ($\Delta = +0.146$, $p_{\text{Holm}} = 0.0001$), line windows without headers significantly outperform AST chunks with headers ($\Delta = -0.093$, $p_{\text{Holm}} = 0.0438$), and strongly outperform

AST chunks without headers ($\Delta = -0.239$, $p_{\text{Holm}} < 0.001$). Following parser grammar expansions, the AST fallback rate was 0.0%, confirming that node-level AST extraction without sibling merging does not surpass sliding windows in requirement evidence retrieval. We pose three open questions for the community: (1) *Can graph-enriched headers bridge the information gap for fine-grained AST chunks?* (2) *How should code-RAG benchmarks weigh syntactic integrity against contextual information density?* (3) *How can requirement evidence retrieval best assist practitioners in verifying codebase capabilities?*

Data Availability

The replication package (corpus code, frozen queries, Tree-sitter parsers, 882 blinded human-labeled pairs, adjudication records, and benchmark scripts) is available at: https://anonymous.4open.science/r/ast-rag-replication-2027-82C5/.

References

[1] G. Antoniol, G. Canfora, G. Casazza, A. D. Lucia, and E. Merlo, "Recovering traceability links between code and documentation," *IEEE Transactions on Software Engineering*, vol. 28, no. 10, pp. 970–983, 2002.

[2] H. Gao, H. Kuang, W. K. G. Assunção, C. Mayr-Dorn, G. Rong, H. Zhang, X. Ma, and A. Egyed, "Triad: Automated traceability recovery based on biterm-enhanced deduction of transitive links among artifacts," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024, pp. 2668–2680.

[3] Y. Zhang, X. Zhao, Z. Z. Wang, C. Yang, J. Wei, and T. Wu, "cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 8106–8116.

[4] X. Wu, J. Gong, G. Jahangirova, and J. Zhang, "How does chunking affect retrieval-augmented code completion? a controlled empirical study," *arXiv preprint arXiv:2605.04763*, 2026.

[5] T. Galimzyanov, O. Kolomyttseva, and E. Bogomolov, "Practical code RAG at scale: Task-aware retrieval design choices under compute budgets," *arXiv preprint arXiv:2510.20609*, 2025.

[6] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[7] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 7212–7225.

[8] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[9] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[10] F. Hu, Y. Wang, L. Du, X. Li, H. Zhang, S. Han, and D. Zhang, "Revisiting code search in a two-stage paradigm," in *Proceedings of the 16th ACM International Conference on Web Search and Data Mining (WSDM)*, 2023, pp. 1066–1074.

[11] D. Zheng, Y. Wang, W. Chen, J. Chen, and Z. Zheng, "CoSTV: Accelerating code search with two-stage paradigm and vector retrieval," in *Proceedings of the 31st Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2024, pp. 1–10.

[12] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.

[13] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking complete code generation with repository-level context," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] Y. Ding, Z. Wang, W. Wang, H. Peng, S. Zou, J. McAuley, and T. Chen, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[15] J. E. Montandon, L. L. Silva, and M. T. Valente, "Identifying experts in software libraries and frameworks among GitHub users," in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 276–287.

[16] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese and multilingual embedding," *arXiv preprint arXiv:2309.07597*, 2023.

[17] C. Buckley and E. M. Voorhees, "Retrieval evaluation with incomplete information," in *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2004, pp. 25–32.

[18] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.